Java 入门笔记

第五课 · Java 选择结构

学习目标	掌握 if、if-else、else-if、嵌套 if 与传统 switch，能根据条件特点选择清晰、安全的分支结构。

目录

本课目标 2

3.8 条件与布尔表达式 2

3.9 简单 if 语句 2

3.10 if-else 语句 3

3.11 else-if 多分支 4

3.12 嵌套 if 4

3.13 综合例题：一元二次方程 4

3.14 switch 语句 5

3.15 if 与 switch 的选择 7

常见错误 7

本课小结 7

自测题 7

整理日期：2026 年 10 月 16 日

> 第四课中的顺序结构会执行所有语句；本课使用 if 和 switch，让程序根据条件选择要执行的分支。

本课目标

- 会用布尔表达式描述判断条件。
- 掌握 if、if-else、else-if 和嵌套 if。
- 掌握传统 switch 的匹配、break 与穿透现象。
- 能根据问题特点选择合适的分支结构。

3.8 条件与布尔表达式

选择结构的条件必须得到 boolean 值，即 true 或 false。

```
age >= 18
score >= 60 && score <= 100
number % 2 == 0
name.equals("Java")
```

注意：赋值运算符是 =，相等比较是 ==；比较字符串内容通常使用 equals，不能依赖 ==。

三元运算符适合从两个简单值中选择一个：

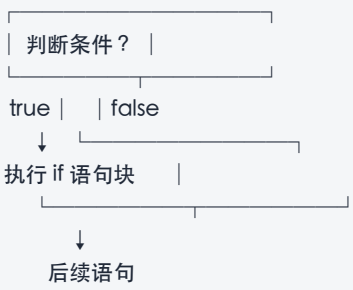
```
int max = a > b ? a : b;
```

当分支内有多条语句或判断层次较多时，应使用 if，不要把三元表达式写得过于复杂。

3.9 简单 if 语句

3.9.1 语法与流程

```
if (条件表达式) {
   条件为 true 时执行的语句;
}
```



3.9.2 三个整数升序排列

相邻位置不满足升序时交换，三个独立 if 按顺序执行。

```
public class SortThreeNumbers {
    public static void main(String[] args) {
        int a = 9, b = 3, c = 6;
        int temp;

        if (a > b) {
            temp = a; a = b; b = temp;
        }
        if (a > c) {
            temp = a; a = c; c = temp;
        }
        if (b > c) {
            temp = b; b = c; c = temp;
        }

        System.out.printf("%d <= %d <= %d%n", a, b, c);
    }
}
```

printf 中 %d 表示整数，%n 表示换行。

3.10 if-else 语句

3.10.1 语法与流程

```
if (条件表达式) {
    条件为 true 时执行的语句;
} else {
    条件为 false 时执行的语句;
}
```

```
          判断条件？
         /        \
       true        false
        ↓           ↓
   执行 if 分支    执行 else 分支
         \        /
           后续语句
```

```
int a = 11;
int b = 23;
if (a > b) {
    System.out.println(a + " 较大");
} else if (a < b) {
    System.out.println(b + " 较大");
} else {
    System.out.println("两数相等");
}
```

3.11 else-if 多分支

当条件有三个或更多互斥结果时，可以使用 else-if 链。程序从上到下判断，执行第一个成立的分支后结束整条结构。

```
if (score < 0 || score > 100) {
    System.out.println("成绩无效");
} else if (score >= 90) {
    System.out.println("优秀");
} else if (score >= 80) {
    System.out.println("良好");
} else if (score >= 60) {
    System.out.println("及格");
} else {
    System.out.println("不及格");
}
```

条件顺序很重要：范围通常从更严格到更宽松排列。如果先写 score >= 60，那么 90 分也会在该分支结束，后面的“优秀”永远无法执行。

3.12 嵌套 if

把一个 if 或 if-else 放进另一个分支，就形成嵌套。适合先判断大类，再判断小类。

```
if (age >= 18) {
    if (hasTicket) {
        System.out.println("允许入场");
    } else {
        System.out.println("请先购票");
    }
} else {
    System.out.println("年龄不符合要求");
}
```

Java 中 else 与最近且尚未配对的 if 配对。始终写大括号并保持缩进，可以避免“悬空 else”造成误解。

3.13 综合例题：一元二次方程

对于 $ax^2 + bx + c = 0$，必须先判断 a，再根据判别式 $\Delta = b^2 - 4ac$ 决定根的情况。浮点数计算存在舍入误差，实际程序可用很小的误差范围代替直接比较 == 0。

```
import java.util.Scanner;

public class QuadraticEquation {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        final double EPSILON = 1e-10;

        System.out.print("请输入 a、b、c：");
        double a = scanner.nextDouble();
        double b = scanner.nextDouble();
        double c = scanner.nextDouble();

        if (Math.abs(a) < EPSILON) {
            System.out.println("a 不能为 0，这不是一元二次方程");
        } else {
            double delta = b * b - 4 * a * c;
            if (delta > EPSILON) {
                double x1 = (-b + Math.sqrt(delta)) / (2 * a);
                double x2 = (-b - Math.sqrt(delta)) / (2 * a);
                System.out.println("两个不相等的实数根：" + x1 + "，" + x2);
            } else if (Math.abs(delta) <= EPSILON) {
                double x = -b / (2 * a);
                System.out.println("两个相等的实数根：" + x);
            } else {
                System.out.println("在实数范围内无解");
            }
        }

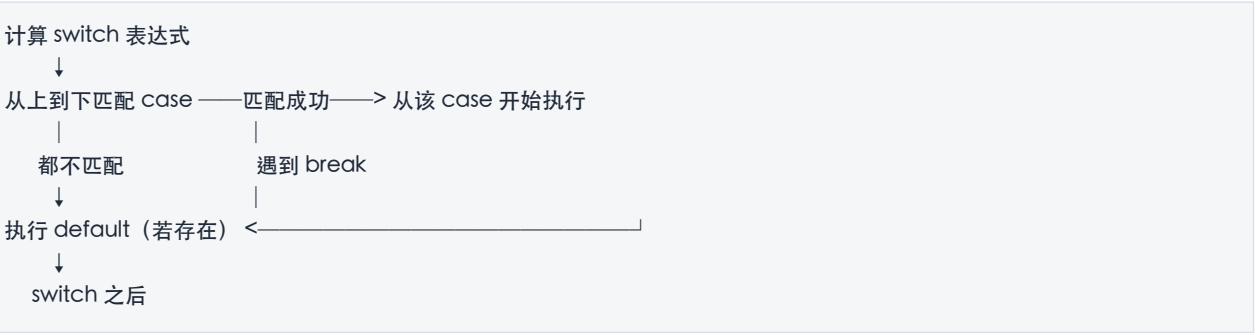
        scanner.close();
    }
}
```

3.14 switch 语句

3.14.1 语法与执行过程

传统 switch 适合把一个离散值与若干固定选项匹配。

```
switch (表达式) {
    case 常量1:
        语句组1;
        break;
    case 常量2:
        语句组2;
        break;
    default:
        默认语句组;
}
```



入门阶段常用的表达式类型包括 byte、short、char、int、对应包装类型、String 和枚举。传统写法不使用 float、double 或 boolean 作为匹配值。

3.14.2 break 与穿透

break 会结束当前 switch。省略 break 后，程序会继续执行后面的 case，这种现象称为穿透。只有确实需要多个 case 共用逻辑时才应利用穿透。

```
int month = 2;
switch (month) {
    case 1:
    case 2:
    case 3:
        System.out.println("第一季度");
        break;
    default:
        System.out.println("不是第一季度");
}
```

3.14.3 成绩等级示例

```
int score = 86;
if (score < 0 || score > 100) {
    System.out.println("成绩无效");
} else {
    switch (score / 10) {
        case 10:
        case 9:
            System.out.println("优秀");
            break;
        case 8:
            System.out.println("良好");
            break;
        case 7:
        case 6:
            System.out.println("及格");
            break;
        default:
            System.out.println("不及格");
    }
}
```

3.15 if 与 switch 的选择

情况	更合适的结构
范围判断、复合逻辑、大小比较	if / else-if
一个值匹配多个固定选项	switch
只有两个简单结果	if-else
分支需要继续判断其他条件	嵌套 if 或整理后的 else-if

常见错误

- 在 if (condition) 后误写分号。
- 把 = 当成 ==，或用 == 比较字符串内容。
- 多分支条件顺序不合理，使后面的分支永远不能执行。
- 忘记 switch 的 break，造成非预期穿透。
- 计算一元二次方程时先除以 2 * a，却没有先检查 a。

本课小结

- 简单 if 决定“是否执行”，if-else 在两个分支中二选一。
- else-if 适合范围与复合条件；嵌套 if 适合分层判断。
- switch 适合一个离散值匹配多个固定选项，传统写法要留意 break。
- 分支结构结束后，程序继续执行后续语句。若需要重复执行，就进入第六课的循环结构。

自测题

1. 为什么成绩等级判断要把 score >= 90 写在 score >= 60 前面？
2. else 会与哪个 if 配对？
3. 什么情况下 switch 的穿透是有意为之？
4. 编写程序：输入年份，判断它是否为闰年。